# CPTS 360 - Computer Memory Systems

Notes rewritten from course material by Professor Monowar Hasan, Washington State University. AI assistance: OpenAI GPT-5.6 Sol.

## 1. RAM and the CPU-Memory Gap

**Main idea:** Fast memory is expensive, so systems combine several storage technologies.

- **SRAM** is faster and more expensive; it is used for cache memory.
- **DRAM** is used for main memory and graphics systems.
- CPU speed has pulled away from DRAM and storage speed.
- Programs bridge this gap by taking advantage of locality.

## 2. Locality

**Main idea:** Programs often reuse recent items and access nearby addresses.

- **Temporal locality:** a recently used item is likely to be used again soon.
- **Spatial locality:** items at nearby addresses are likely to be used close together in time.
- Sequential array access has good spatial locality.
- Repeatedly updating the same variable has good temporal locality.
- C stores two-dimensional arrays in row-major order, so row-by-row access is usually more local than column-by-column access.

```
sum = 0;
for (i = 0; i < n; i++)
    sum += a[i];

// a[i]: spatial locality
// sum:  temporal locality
```

## 3. Memory Hierarchy

**Main idea:** A hierarchy makes a large storage system appear faster by placing frequently used data near the CPU.

- Higher levels are smaller, faster, and more expensive per byte.
- Lower levels are larger, slower, and cheaper per byte.
- Typical order: registers, L1 cache, L2 cache, L3 cache, main memory, local storage, remote storage.
- Each faster level acts as a cache for the larger level below it.
- The hierarchy works because well-written programs exhibit locality.

## 4. Cache Basics

**Main idea:** A cache holds a subset of blocks copied from a larger, slower memory.

- A **hit** occurs when the requested block is already in the cache.
- A **miss** fetches the block from the next level and stores it in the cache.
- A **cold miss** occurs because the cache starts empty.
- **Placement policy** decides where a block can go.
- **Replacement policy** chooses the victim block when space is needed.
- Examples include random replacement and least recently used (LRU).

## 5. Cache Organization and Address Fields

**Main idea:** A cache is divided into sets, lines, and fixed-size blocks.

- S = 2^s sets; E = 2^e lines per set; B = 2^b bytes per block.
- Cache data capacity: **C = S x E x B bytes**.
- Each line contains a valid bit, tag, and data block.
- An address is split into **tag | set index | block offset**.
- The set index selects a set, the tag identifies a matching line, and the offset selects bytes inside the block.
- A read is a hit only when a line in the selected set is valid and its tag matches.

```
address:  [ tag | set index | block offset ]

set index    -> choose set
tag          -> check matching valid line
block offset -> choose data inside block
```

## 6. Direct-Mapped and Set-Associative Caches

**Main idea:** Associativity controls how many cache lines are available in each set.

- A **direct-mapped cache** has E = 1, so each set has one line.
- On a tag mismatch, the direct-mapped line is replaced.
- An **E-way set-associative cache** compares the requested tag against every line in the selected set.
- If no line matches, one line in that set is selected for eviction.
- More lines per set can prevent some conflicts seen in a direct-mapped cache.

## 7. Cache Writes

**Main idea:** A write policy decides when updated data reaches the next memory level.

- **Write-through:** update memory immediately on a write hit.
- **Write-back:** delay the memory update until the line is replaced; a dirty bit records the difference.
- **Write-allocate:** load a missed block into cache, then update it.
- **No-write-allocate:** send a write miss directly to memory without loading the block.
- Typical pairings are write-through with no-write-allocate and write-back with write-allocate.

## 8. Cache Performance

**Main idea:** Even a small reduction in miss rate can greatly reduce average access time.

- **Miss rate = misses / accesses = 1 - hit rate.**
- **Hit time:** time to find and deliver data already in the cache.
- **Miss penalty:** extra time required after a cache miss.
- With a 1-cycle hit time and 100-cycle miss penalty, 97% hits gives 4 cycles on average.
- Under the same assumptions, 99% hits gives 2 cycles on average.

```
average access time = hit time + miss rate * miss penalty

97% hits: 1 + 0.03 * 100 = 4 cycles
99% hits: 1 + 0.01 * 100 = 2 cycles
```

## 9. Writing Cache-Friendly Code

**Main idea:** Access order can change performance even when two programs perform the same overall work.

- Focus optimization on inner loops of frequently used functions.
- Reuse variables to gain temporal locality.
- Use stride-1 access patterns to gain spatial locality.
- Walking across a row of a row-major C array accesses successive elements.
- Walking down a column accesses distant elements and can produce a much higher miss rate.
- The course copy example took 4.3 ms row-first and 81.8 ms column-first on the stated system.

```
// Row-first: better spatial locality
for (i = 0; i < 2048; i++)
    for (j = 0; j < 2048; j++)
        dst[i][j] = src[i][j];
```

## 10. Matrix Multiplication Loop Order

**Main idea:** Changing loop order changes memory access patterns and cache misses without changing the mathematical result.

- The analysis assumes 32-byte blocks, four doubles per block, large matrices, and local variables stored in registers.
- **ijk** and **jik**: row-wise A, column-wise B, fixed C; 1.25 misses per inner iteration.
- **kij** and **ikj**: fixed A, row-wise B and C; 0.5 misses per inner iteration.
- **jki** and **kji**: column-wise A and C, fixed B; 2.0 misses per inner iteration.
- In this comparison, kij and ikj have the best locality.

```
// Cache-friendlier order from the course comparison
for (i = 0; i < n; i++)
    for (k = 0; k < n; k++) {
        r = a[i][k];
        for (j = 0; j < n; j++)
            c[i][j] += r * b[k][j];
    }
```

**Self-study listed in the course material:** CSAPP 6.4.2 - Conflict Misses in Direct-Mapped Caches.